

Project 0 — Bake-Off

CS496 AI Engineering

Dr Abdeldjalil Labed

Mediterranean Institute of Technology

Fall 2026



Outline

The brief

The task

The models

What to measure

What you hand in

The brief

Choose a **small task that has a correct answer**. Run it on **three models**: one top API model, one cheap API model, and one open-weights model **that you run yourself**. Use **at least 50 test items**.

You hand in: a **two-page report** with cost per 1k requests, p50/p95 latency, accuracy, and one paragraph that says **which model you would choose** and **when you would change your choice**.

- **Time:** Week 1
- **Weight:** 5% of the final grade
- **Team:** max 4 students, same team all term
- **Deadline:** repo frozen before the first session of Week 2
- **Demo:** first session of Week 2

Objective

The whole course asks one question: “**Which option is better, and how do you know?**” This project is a simple version of that question.

At the end of the week, every team member can:

1. **Build a test set:** 50 or more items where the correct answer is clear.
2. **Score automatically:** a function that says right or wrong. No human, no LLM.
3. **Compare fairly:** same prompt, same items, same settings, same parser for all three models.
4. **Measure three things:** accuracy, cost per 1k requests, and p50 / p95 latency, from real runs.
5. **Run a model yourself:** an open-weights model on your own hardware.
6. **Make a choice:** which model you would use, and what would make you change.

A good task: say yes to all six

1. **The correct answer is clear.** Two careful people give the same label.
2. **Scoring is automatic.** `score(output, expected)` returns right or wrong.
3. **One model call per item.** No agents, no retrieval, no chat history.
4. **Inputs are short.** A few hundred to a few thousand tokens each.
5. **You can make 50+ items in two days,** and two people check every label.
6. **The models will not all get 100%.** If they all do, you learn nothing.

If you need an LLM to check the answer, it is a Project 1 task, not a Project 0 task.

Task ideas

Pick one, or propose your own. Get the instructor's OK by Day 2.

- **Read fields from invoices**
exact match for each field
- **Write SQL from a question**
run both queries, compare results
- **Write a regex from a spec**
it must pass all 20 test strings
- **Translate Arabic headlines to English**
chrF score + a human check
- **Expand medical abbreviations**
exact match, fixed word list
- **Label a code diff** (feat / fix / refactor / docs)
true label from the repo history
- **Sort bank transactions into categories**
exact match, 12 categories
- **Classify legal clauses**
exact match on the class
- **Unit-conversion word problems**
number within tolerance + right format
- **Read fields from job ads**
exact match for each field

Three models, same conditions

The three models

- **Top API model:** the best model a provider sells.
e.g. Claude Opus 5, the top GPT or Gemini model
- **Cheap API model:** the small, fast model.
e.g. Claude Haiku 4.5, “mini” or “Flash” models
- **Open-weights model, run by you:** Llama, Qwen, Gemma, Mistral, Phi (1B–14B) with Ollama, llama.cpp, or vLLM.
Calling an open model through someone else’s API does not count.

Keep the same for all three

- The same prompt, exactly
- Temperature 0, same output-token limit
- The same output format and the same parser
- The same 50 items, in the same order
- Write down the exact model name and the date

Not allowed: a different prompt per model, removing items a model failed, fixing outputs by hand.

Accuracy, latency, cost

Accuracy

- Report **n / 50**, not only a percentage
- A parse error, a refusal, or a timeout counts as **wrong**. Show them in their own column
- Save **one result per item** in a CSV. Build the table from that file
- Show 3 wrong answers per model

Latency

- Time from sending the request to getting the full answer, in ms
- At least 50 calls per model, **one after the other**
- Report **p50 and p95**. Never only the average
- For your own model: also tokens per second and the hardware

Cost per 1k requests

- API: token counts from the **usage field** of the real answers $\times$ the list price
- Your own model: hardware cost per hour $\div$ requests per hour, plus your time
- Today, and at **100 $\times$ more traffic**
- The volume where API and self-hosted **cost the same**

What you hand in

The repo

- `README.md`: one setup command, one run command, the results table, a Contributions section (one entry per member)
- `data/items.jsonl` + a short labelling note
- `src/`: `prompt`, `run.py`, `score.py`, `cost.py`
- `results/per_item.csv`, `summary.csv`, hardware note
- `report.pdf` (2 pages), `postmortem.md` (1 page)
- No API keys in the repo

The two pages

1. The task and one example item
2. The data: where it came from, how you checked labels, dev/test split
3. Setup: 3 model names, settings, prompt, hardware
4. **Results table** + 3 wrong answers per model
5. **Your choice, and when you would change it**
6. Cost at 100× traffic and the break-even volume

Postmortem: what went wrong and what you learned. A postmortem with no problems loses points.

Rules for every project: runs from a clean clone · has an eval · has a cost model · has a postmortem · you can explain every line.